Microprocessor Systems

# Basics of Assembly Programming

Muhammad Tahir
Kashif Javid

Department of Electrical Engineering
University of Engineering and Technology Lahore

# Contents

# Introduction

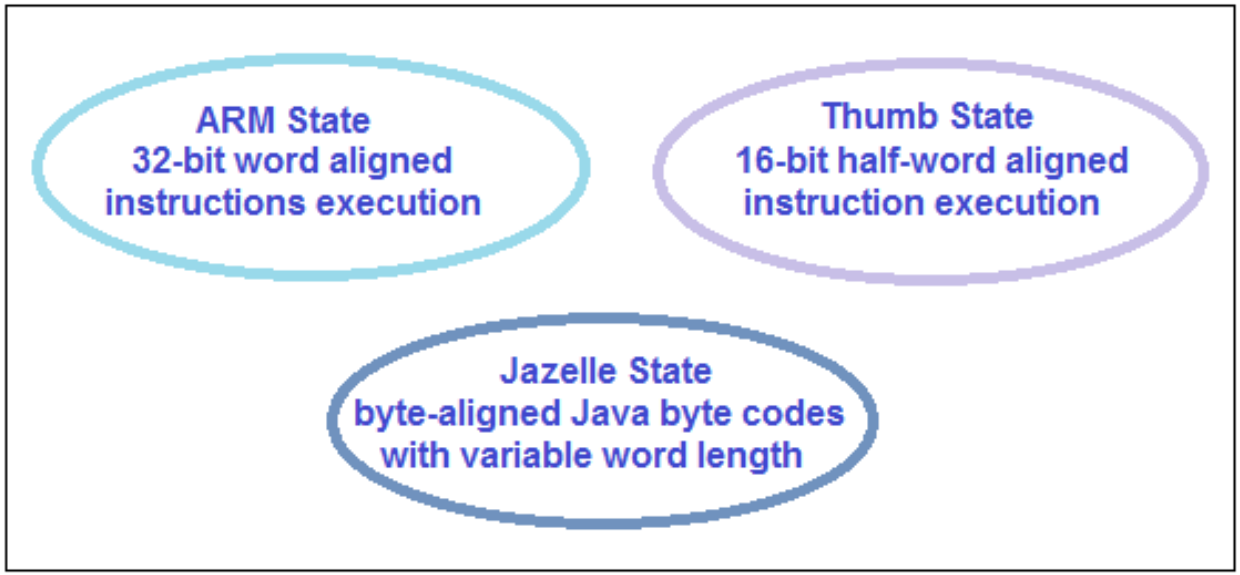



Figure: ARM, Thumb and Jazelle States

The processors that supported both 32-bit as well as 16-bit instructions required to switch between ARM state and Thumb state incurring an additional overhead of state switching.

# Thumb 2


Figure: Thumb 2 instruction set

- No state switching is required which results in an improved execution performance and saves instruction memory space as well.
- Cortex-M processor uses Thumb2 instruction set

# Cont'd:

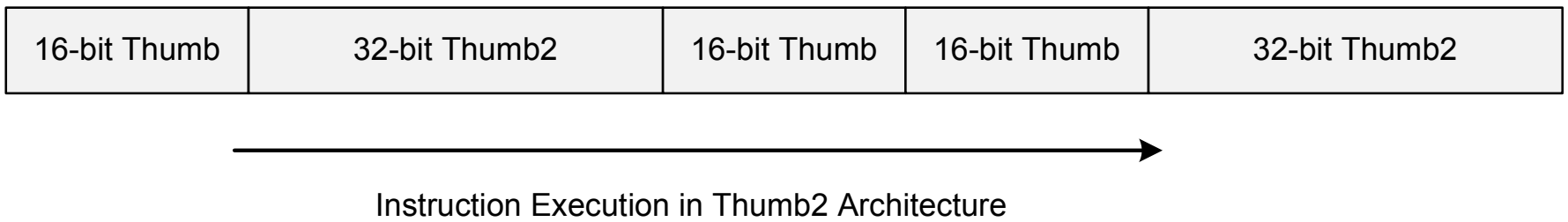



Figure: The 16-bit and 32-bit mixed instruction execution in Thumb2 ISA.

- Thumb2 instruction set allows free intermixing of 32 and 16 bit instructions.
- Cortex-M3 does not implement all of the instructions provided by the Thumb2 instruction set.

# Why Use Assembly Language?

- An assembly language program provides complete as well as precise control of the available hardware resources.
- When writing a program in a high level language, the user is mostly unaware of the underlying hardware capabilities, which can lead to highly inefficient implementation.
- Programming in assembly language also gives the user complete understanding of the system architecture.

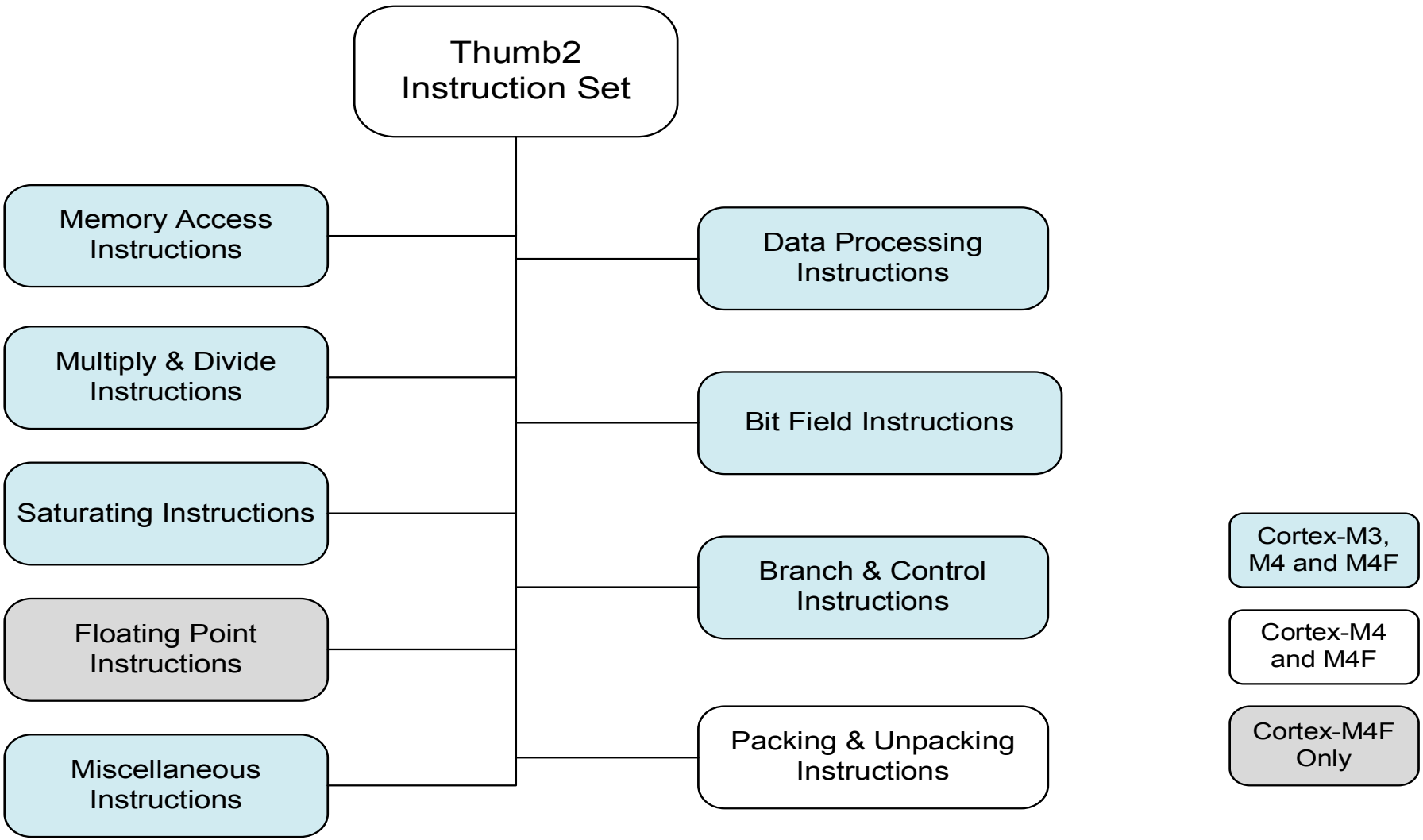
# Syntax of an assembly program

```
label opcode operand_1, operand_2, ..., operand_n ; Comments
```

Listing 1: Syntax of an assembly program.

| | | |
|---|---|---|
| label | → | For determining address of instruction |
| opcode | → | Represents instruction's machine code |
| operands | → | No. of operands depends upon instruction |
| Comments | → | For better human understanding |

# Instruction Set Groups



# Add Instruction

## ADD instruction illustration

```
ADD    R1, R2,#10          ; R1 = R2 + 10
ADD    R6, R5, R3          ; R6 = R5 + R3
ADD    R1 , #0x6           ; R1 = R1 + 0x6
ADD    R4 , R2             ; R4 = R4 + R2
ADDS   R6 , R5 , R3        ; R6 = R5 + R3 , flags are updated
```

## MOV instruction illustration

```
MOV R2, #0x123      ; move immediate value of 0x123 to R2
MOV R4, #'A'        ; move ASCII value of A(i.e. 0x41) to R4
MOV R7, R3          ; move the contents of R3 to R7
```

# Load Store Instructions

```
LDR  R2, [R1]       ;load R2 with data from memory pointed to
                    ;by R1
LDR  R0, =NUM1      ; load R0 with the value of constant NUM1
                    ; (this constant might represent
                    ;  the memory address)
STR  R4, [R3]       ; store R4 to a memory location addressed
                    ; by R3
```

Listing 2: Memory access instructions.

# Branch Instructions

```
loop1   LDR R2, [R1]        ; load R2 with data from memory
                            ;  pointed to by register R1
...
CBZ R5, label1              ; Compare R5 with zero. If
                            ; comparison result is true
                            ;then branch to label1
...
B    loop1                  ; jump to the memory location
                            ;  labeled as loop1
label1
MOV R3, #0x034
```

Listing 3: Simple memory access instructions.

# Example Program

- Compute the Sum of first five even numbers

**Initializions**

sum$\leftarrow$ 0

loop counter $\leftarrow$ 5

Even number $\leftarrow$ 2

**Computing Sum**

**repeat** till counter is not zero

sum $\leftarrow$ sum + Even number

Even number $\leftarrow$ Even number + 2

Decrease counter

# Example Program Cont'd

```
MOV R0, #0          ; R0 will accumulate the sum
MOV R1, #2          ; R1 will have the updated even number
MOV R2, #5          ; the counter for the loop

lbegin
CBZ R2, lend        ; If R2 != 0 continue with the next
                    ; instruction
ADD R0, R1
ADD R1, #2
SUB R2, #1
B lbegin            ; branch unconditionally to lbegin
lend
```

Listing 4: A simple program calculating the sum of the first five even numbers.

# Assembly Source File

- An assembly source file is a collection of
  - (i) assembly language instructions
  - (ii) assembler directives
  - (iii) if required, may also include macro directives.
- The assembler translates assembly language source files to machine understandable object file.
- Object file is used by the linker along with linker command file (based on linker directives) and any libraries (if required) to generate the executable.

# Assember Directives

- Assembler directives are pseudo-opcodes or pseudo-operations performed by an assembler.
- They are not part of the assembly instruction set.
- They are executed during the assembling process at compilation time
  (assembly instructions are executed at runtime).

# Different Assember Directives

- AREA
- Size Directives: SPACE, DCB, DCW, DCD
- EQU: Symbolic name to a numeric constant
- ENTRY: Declares an entry point to a program
- END: Informs the assembler that it has reached the end of a source file

# Different Assember Directives Cont'd:

- GLOBAL and EXPORT: Direct the assembler that the associated labels are not defined in the current assembly program file and need to be looked into some other object files (generated from other source files) or libraries
- THUMB: Instructs the assembler to interpret subsequent instructions as Thumb or Thumb2 instructions
- PRESERVE8: Specifies that the current file preserves 8-byte alignment of the stack

# AREA Directive

The AREA directive is used to instruct the assembler for a new code or data section.

```
\textrm{AREA}     sectionname  {,attr} {,attr} ...
 AREA     sectionname  {,attr} {,attr} ...
```

Listing 5: Examples of AREA section directive.

- *sectionname* is the name assigned by the user to a particular data or code section.
  - (i) |.text| is used to mention code sections generated by the C compiler
  - (ii) STACK indicates the stack area, which is in the RAM

# AREA Directive Illustration

```
AREA    Mydata, DATA, READWRITE  ; Define a data section
; with section name Mydata

AREA    STACK, NOINIT, READWRITE, ALIGN=3
AREA    |.text|, CODE, READONLY, ALIGN=2
```

Listing 6: Examples of AREA section directive.

# AREA Directive Attributes

| Attribute | Description |
|---|---|
| READONLY | Indicates that this section must not be written to. This is the default for code areas. |
| NOINIT | Use of this attribute indicates that the data section is uninitialized, or initialized to zero. A section with this attribute contains only space reservation directives SPACE or DCB, DCD, DCDU, DCQ, DCQU, DCW, or DCWU with initialized values of zero. You can decide at link time whether an area is uninitialized or zero initialized. |
| READWRITE | Indicates that this section can be read from and written to. This is the default for Data areas. |
| DATA | Contains data, not instructions. READWRITE is the default. |
| CODE | Contains machine instructions. READONLY is the default. |
| ALIGN=*number* | By default, sections are aligned on a four-byte boundary. The *number* can have any integer value from 0 to 31. The section is aligned on a $2^{number}$ byte boundary. For example, if expression is 10, the section is aligned on a 1 KB boundary. |

# Reset Sequence and Memory Map

An assembly program needs to perform the following sequence of activities to ensure that the user application program is executed properly.

- Define the stack size and reserve appropriate memory space for stack.
- Define the reset vector.
- Write the reset handler code to perform any system related initializations and then make a jump to the user application program.

# Reset Sequence

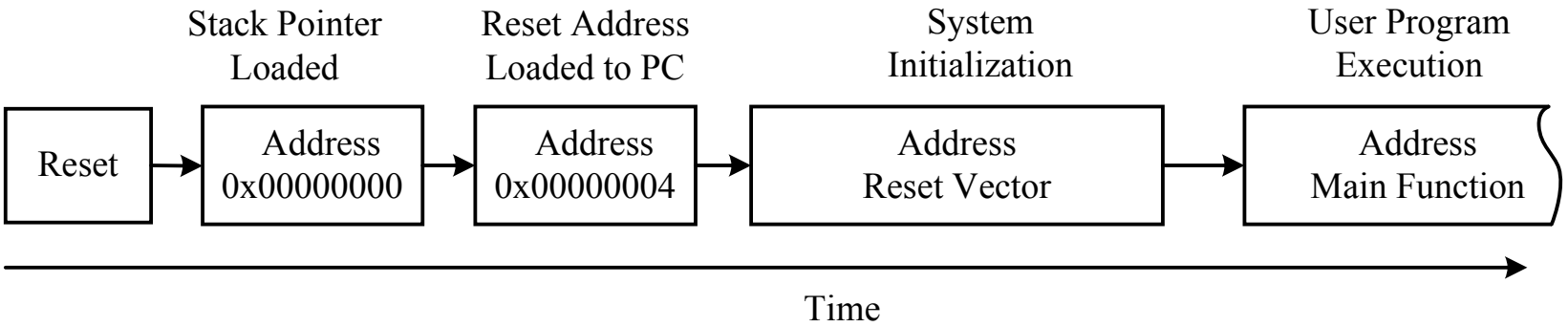
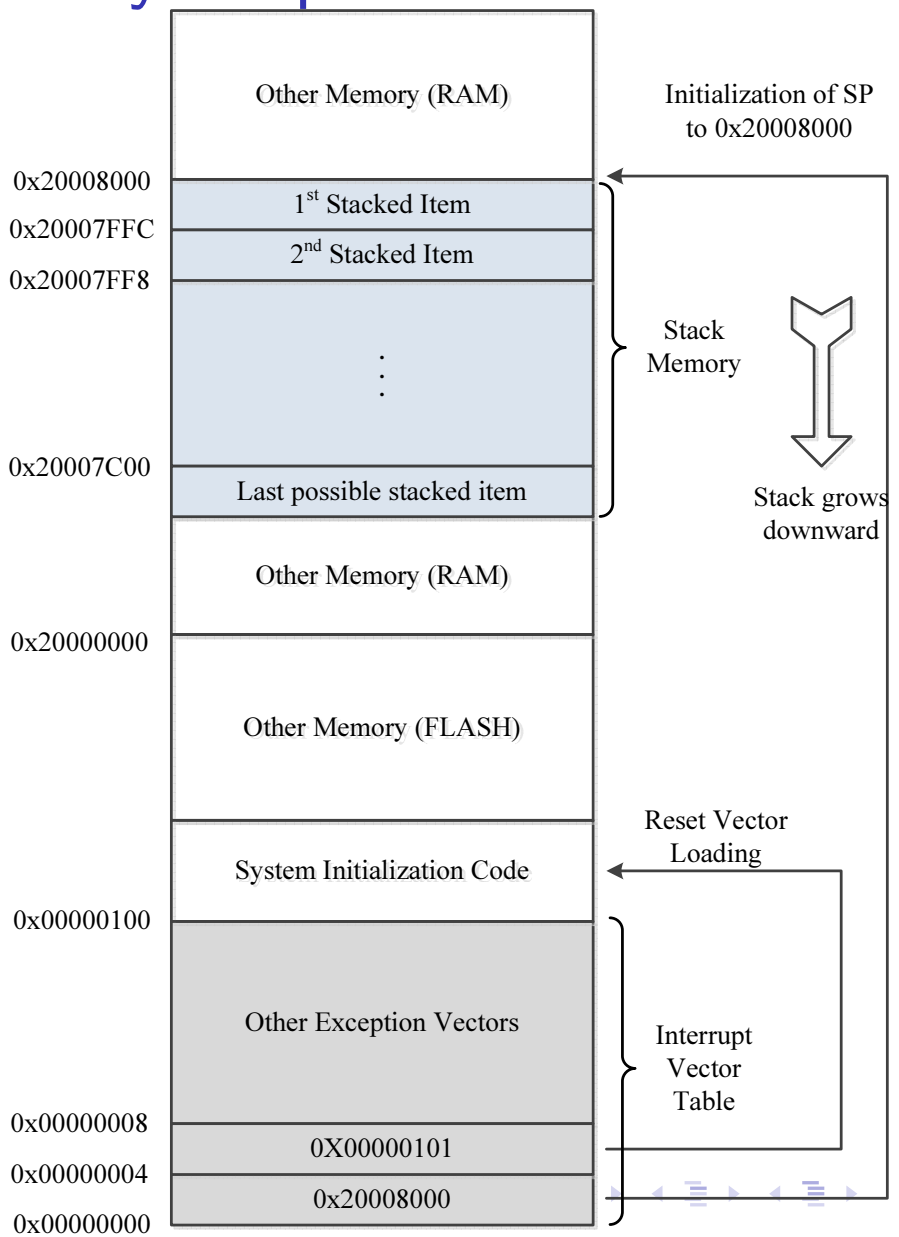



Figure: Reset sequence of Cortex-M processor.

- Only MSP is initialized on the reset and the value of PSP is undefined on reset.
- The bit field T in the execution program status register (EPSR) is set to '1' to mark the Thumb state.
- The link register LR is reset to 0xFFFFFFFF.

# Memory Map

Reset sequence of Cortex-M mapped to memory address map.



# Complete Assembly Language Program

```
 THUMB                    ; Marks the THUMB mode of operation

StackSize  EQU  0x00000100 ; Define stack size of 256 byes

 AREA STACK, NOINIT, READWRITE, ALIGN=3
MyStackMem  SPACE StackSize

 AREA     RESET, READONLY
 EXPORT   __Vectors
__Vectors
 DCD MyStackMem + StackSize ; stack pointer for empty stack
 DCD Reset_Handler          ; reset vector

 AREA     MYCODE, CODE, READONLY
 ENTRY
 EXPORT Reset_Handler
Reset_Handler
 MOV R0, #0                 ; Initial value of sum
 MOV R1, #2                 ; First even number
 MOV R2, #5                 ; Counter for the loop iterations
```

```
lbegin
 CBZ R2, lend          ; Terminate loop if counter is zero
 ADD R0, R1            ; Build the sum
 ADD R1, #2            ; Generate next even number
 SUB R2, #1            ; Decrement the counter
 B lbegin
lend
 END
```

Listing 7: Complete assembly program calculating the sum of the first five even numbers.

# Multiplication Assembly Code

```
 ; This ARM Assembly language program multiplies two
 ; positive numbers by repeated addition.

 THUMB                       ; Marks the THUMB mode of operation
StackSize EQU 0x00000100 ; Define stack size to be 256 byes

 ; Allocate space for the stack.
 AREA   STACK, NOINIT, READWRITE, ALIGN=3
StackMem   SPACE   StackSize

 ; Initialize the two entries of vector table
 AREA   RESET, DATA, READONLY
 EXPORT   __Vectors
__Vectors
 DCD   StackMem + StackSize ; SP value when stack is empty
 DCD   Reset_Handler         ; reset vector
 ALIGN
```

```
 ; Data Variables are declared in DATA AREA
 AREA     ROMdata, DATA, READONLY
X     DCD    10
Y     DCD     3

 AREA     RAMdata, DATA, READWRITE
PRODUCT DCD   0          ;accumulates  product of X and Y

 ; The user code (program) is placed in CODE AREA
 AREA     |.text|, CODE, READONLY, ALIGN=2
 ENTRY                  ; ENTRY marks the starting point
                        ; of the code execution
 EXPORT   Reset_Handler
```

```
Reset_Handler                ; User Code Starts from next line
 LDR R0, =X                  ; load the address of X in R0
 LDR R1, =Y                  ; load the address of Y in R1
 LDR R2, =PRODUCT            ; load the address of PRODUCT in R2

 LDR R3, [R0]                ; load the value of X in R3
 LDR R4, [R1]                ; load the value of Y in R4
 LDR R5, [R2]                ; load the value of PRODUCT in R5

Lbegin
 CBZ R4, Lend
 ADD R5, R3
 SUB R4, #1
 B Lbegin
Lend
 STR R5, [R2]                ; store the product back to PRODUCT
 ALIGN
 END                         ; End of program, matched with ENTRY
```

Listing 8: Assembly program example for multiplication.

# Instruction Encoding

Instruction encoding is the process of assigning a unique binary codeword to each assembly instruction. One of the main jobs of an assembler is to translate each assembly instruction to an equivalent codeword.
Two type of encodings are discussed:

- 16-bit Instruction Encoding
- 32-bit Instruction Encoding

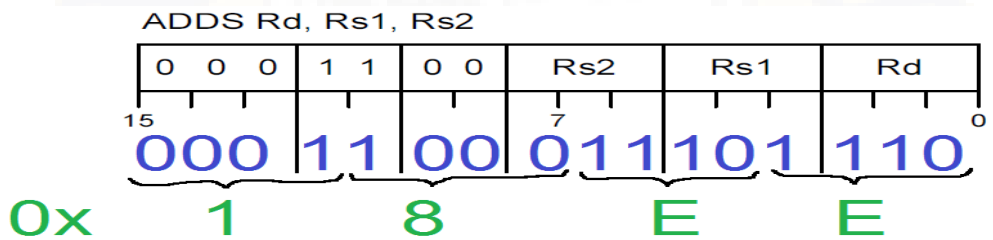
# Instruction Encoding Dependency

| Operand 1 | Operand 2 | Encoding |
|---|---|---|
| Register R0 to R7 | Register R0 to R7 | 16 bit |
| Register R0 to R7 | Immediate value is limited to 3-bit (for different) or 8/12-bit (for same) source-destination registers | 16 bit |
| Register R8 to R12 | Register R0 to R7 | 32 bit |
| Register R0 to R7 | Register R8 to R12 | 32 bit |
| Register R0 to R7 | Immediate value exceeds 3-bit or 8/12-bit | 32 bit |

# 16-bit vs 32-bit Instruction Encoding

## ADDS Rd,Rs1, Rs

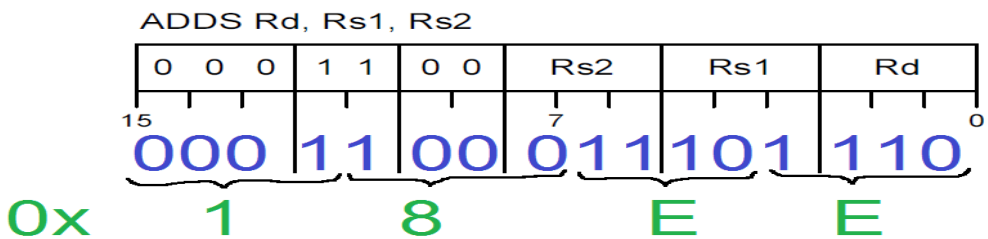
```
ADDS R6, R5, R3   ;R6 = R5 + R3,Instruction is 16-bit encoded
```



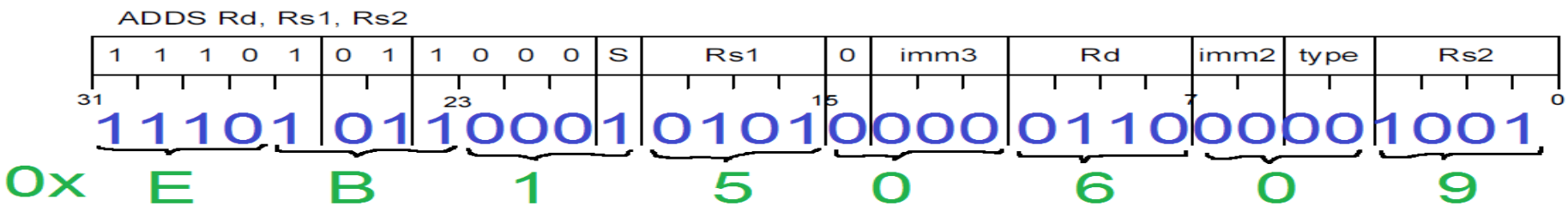
# 16-bit vs 32-bit Instruction Encoding

## ADDS Rd,Rs1, Rs

```
ADDS R6, R5, R3   ;R6 = R5 + R3,Instruction is 16-bit encoded
```

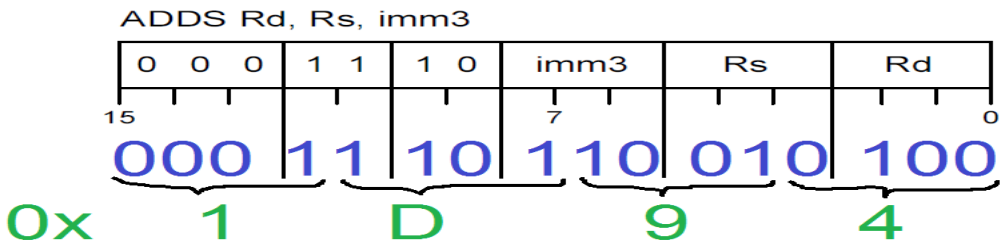



```
ADDS R6, R5, R9   ;R6 = R5 + R9,Instruction is 32-bit encoded
```



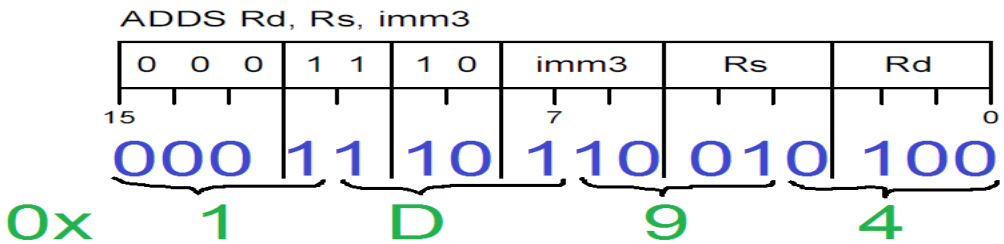
# ADDS Rd,Rs, imm

```
ADDS R4, R2, #6   ;R4 = R2 + 6,Instruction is 16-bit encoded
```

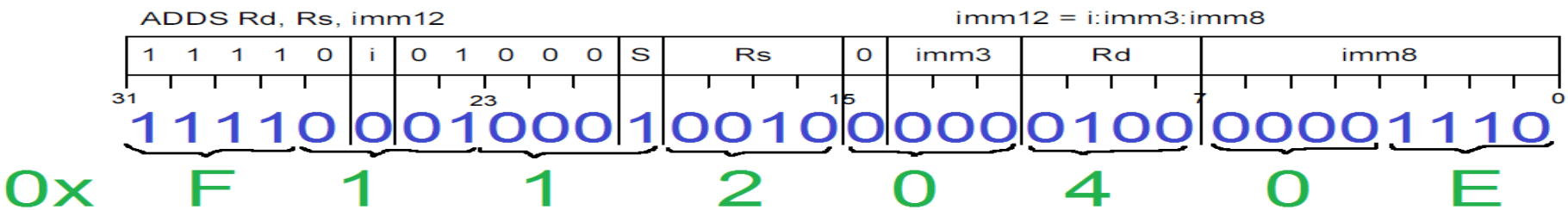


# ADDS Rd,Rs, imm

```
ADDS R4, R2, #6   ;R4 = R2 + 6,Instruction is 16-bit encoded
```

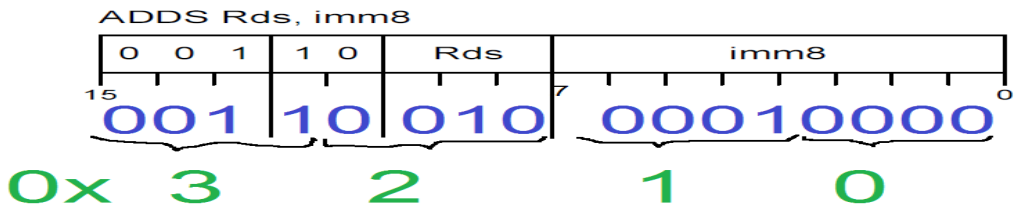



```
ADDS R4, R2, #14  ;R4 = R2 + 14,Instruction is 32-bit encoded
```



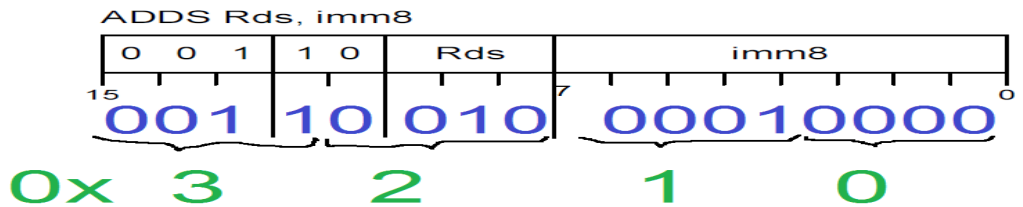
# ADD Rds, imm

```
ADDS R2, #16      ;R2 = R2 + 16, Instruction is 16-bit encoded
```

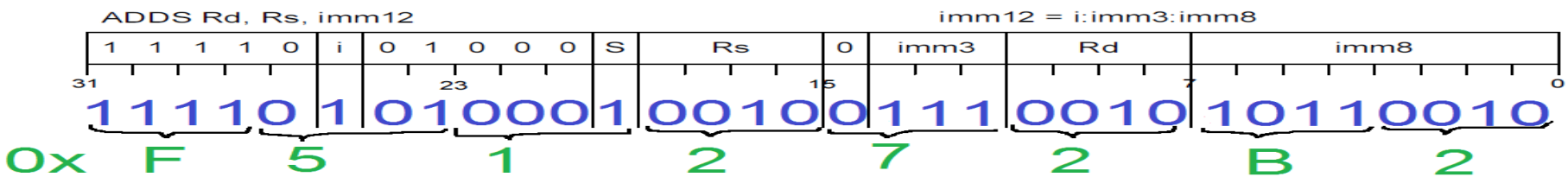


# ADD Rds, imm

```
ADDS R2, #16      ;R2 = R2 + 16, Instruction is 16-bit encoded
```

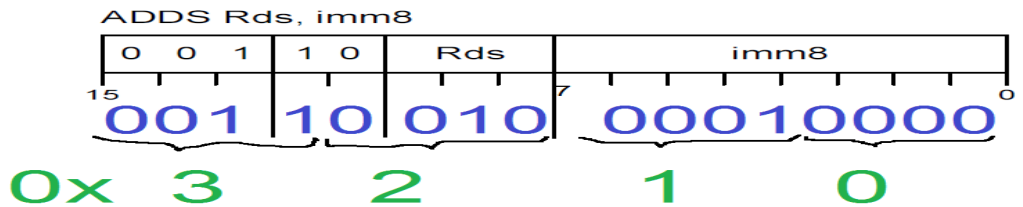



```
ADDS R2, #356     ;R2 = R2 + 356, Instruction is 32-bit encoded
```

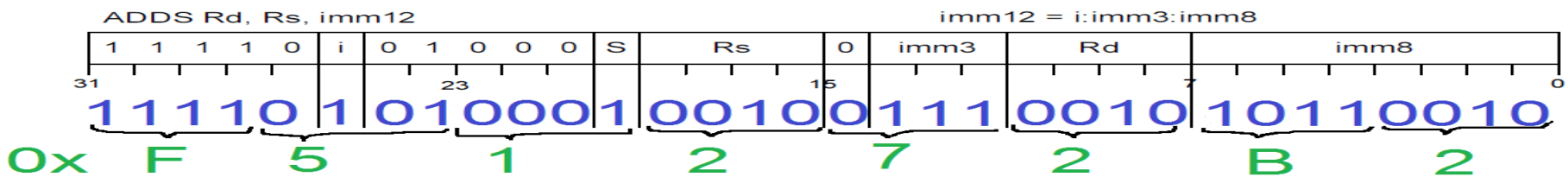


# ADD Rds, imm

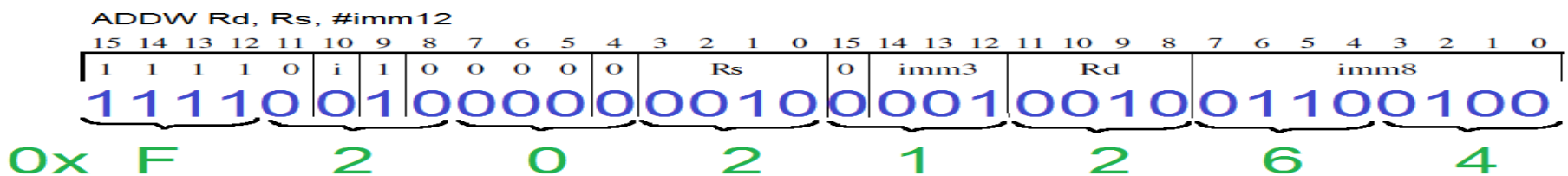
```
ADDS R2, #16      ;R2 = R2 + 16, Instruction is 16-bit encoded
```




```
ADDS R2, #356     ;R2 = R2 + 356, Instruction is 32-bit encoded
```




```
ADDW R2, #356     ;R2 = R2 + 356, Instruction is 32-bit encoded
```



# 12-bit immediate value for ADD{S}

- For values which can be encoded in 8 bits, the 12-bit immediate is just zero-extended immediate value i.e. For imm=0-255 $\Rightarrow$ imm12= 0000imm
- For values greater than 255, the 12-bit immediate is modified as Thumb expanded Immediate value.

| | |
|---|---|
| In binary 356 is | 000101100100 |
| Modified 12-bit immediate value is | 111110110010 |

See article 5.1.2 of text book and Articles 3.3.1 and 4.2 of ARM Architecture Reference Manual Thumb-2 Supplement

# Visualizing Instruction Encoding

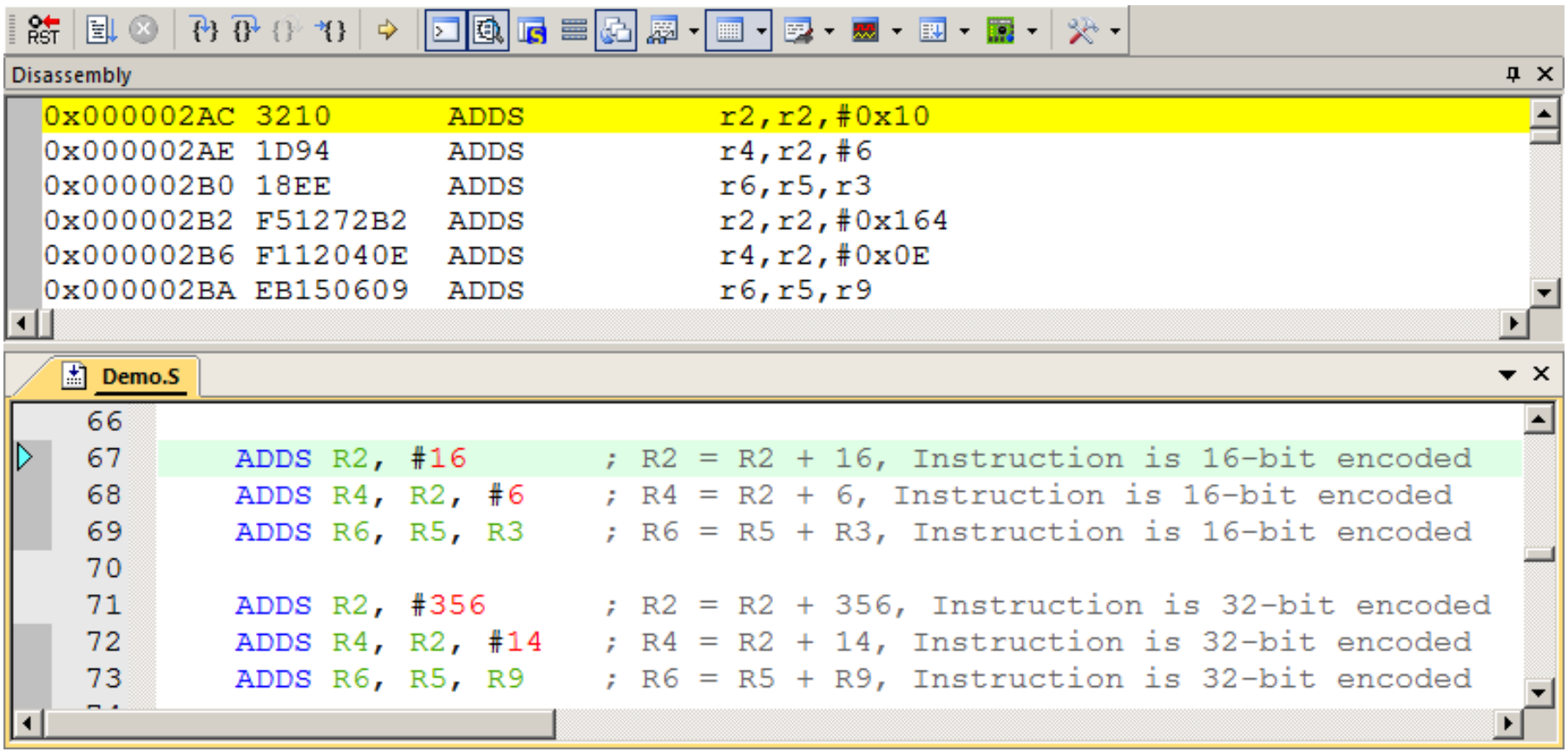
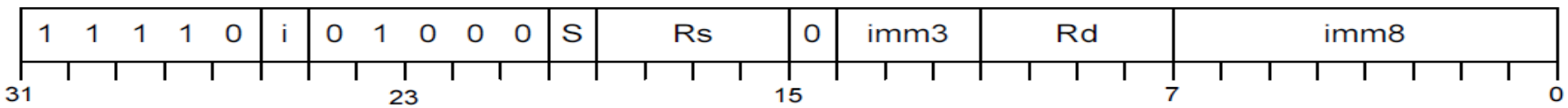



Figure: Illustration of instruction encoding.

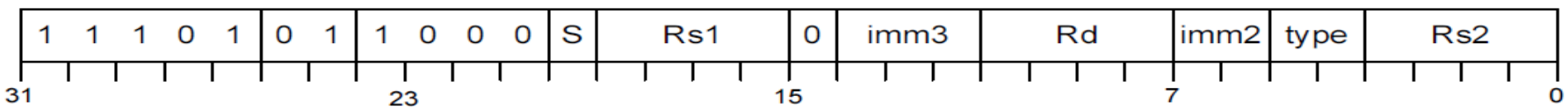
# 32-Instruction Encoding: Thumb2 vs RISC-V

## 32-bit Thumb2 ADD Instruction for immediate and register operands

ADD{S} Rd, Rs, imm12 — imm12 = i:imm3:imm8

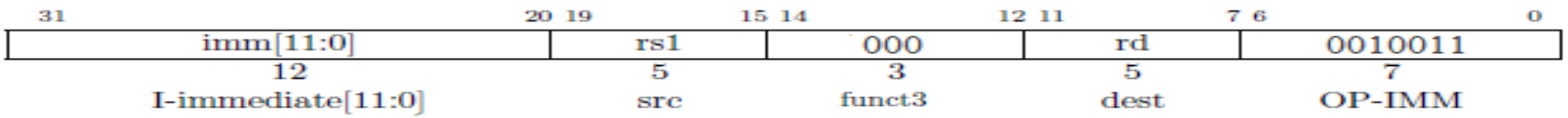



ADD{S} Rd, Rs1, Rs2




## 32-bit RISC-V ADD Instruction for immediate and register operands

ADDI rd, rs1, imm12




ADD rd, rs1, rs2